PRACTICAL TUTORIAL HANDBOOK

Git

Version Control System

Dari Dasar hingga Kolaborasi Tim

Init & Clone	Commit & Stage
Branching	Merge & Conflict
Kolaborasi Tim	Gitmoji & Conv. Commits

Adhi Rahmadian
@adhirahmadian

Daftar Isi

1 Pengenalan Git
1.1 Apa itu Git?
1.2 Instalasi Windows
1.3 Konfigurasi Awal
2 Konsep Dasar
2.1 Repository
2.2 Tiga Area Git
2.3 Alur Kerja
3 Perintah Dasar
3.1 Init & Clone
3.2 Add & Commit
3.3 Status & Log
3.4 Diff
4 Branching & Merging
4.1 Branch Commands
4.2 Strategi Tim
4.3 Merge
4.4 Resolusi Conflict
5 Remote Repository
5.1 GitHub Setup
5.2 Push & Pull
5.3 Fetch vs Pull
5.4 Pull Request
6 Kolaborasi Tim
6.1 Workflow Tim
6.2 .gitignore
6.3 Git Stash
7 Undo & Recovery
7.1 git restore
7.2 git revert
7.3 git reset
7.4 Panduan
8 Good Practice: Gitmoji & Conventional Commits
8.1 Conventional Commits
8.2 Gitmoji
8.3 Panduan Commit
8.4 Workflow Lengkap
9 Referensi Perintah
10 Bibliografi

Bab 1 — Pengenalan Git

1.1 Apa itu Git?

Git adalah sistem version control terdistribusi (DVCS) yang diciptakan Linus Torvalds pada 2005. Git memungkinkan banyak orang bekerja pada proyek yang sama secara bersamaan tanpa saling menimpa pekerjaan satu sama lain. Setiap developer memiliki salinan lengkap seluruh riwayat proyek di komputer lokal mereka.

Keunggulan	Deskripsi
Riwayat Lengkap	Setiap perubahan tersimpan permanen. Kembali ke versi manapun kapanpun
Branching Murah	Buat branch baru hampir tanpa biaya. Eksperimen bebas risiko
Kolaborasi	Banyak developer bekerja paralel, gabungkan perubahan dengan aman
Terdistribusi	Setiap clone adalah backup lengkap. Tidak ada single point of failure

1.2 Instalasi di Windows

1	**Download** https://git-scm.com/download/win
2	**Jalankan Installer** Centang Git Bash Here dan Git GUI Here
3	**Pilih Editor** VS Code direkomendasikan sebagai default editor
4	**PATH Setting** Pilih "Git from the command line and also from 3rd-party software"
5	**Verifikasi** Buka Git Bash: git --version

1.3 Konfigurasi Awal

```
# Wajib dijalankan sekali setelah instalasi
git config --global user.name "Nama Anda"
git config --global user.email "email@contoh.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --list   # verifikasi semua konfigurasi
```

Bab 2 — Konsep Dasar Git

2.2 Tiga Area Git

Area	Deskripsi	Cara Masuk
Working Directory	File yang Anda edit. Perubahan belum diketahui Git	Edit file di editor
Staging Area (Index)	Transit: pilih perubahan yang akan di-commit	git add
Repository (.git/)	Database permanen semua snapshot commit	git commit

Info: Alur dasar: edit file → git add → git commit → git push

Bab 3 — Perintah Dasar Git

3.1 Init & Clone

```
git init                          # repo baru dari direktori
git clone <url>                   # clone dari remote
git clone -b nama-branch <url>    # clone branch tertentu
```

3.2 Add & Commit

```
git status                        # wajib sebelum add
git add nama-file.py              # stage satu file
git add .                         # stage semua perubahan
git commit -m "pesan commit"
git commit --amend -m "perbaiki"  # ubah commit terakhir
```

3.3 Status & Log

```
git status
git log --oneline
git log --oneline --graph --all   # dengan visualisasi branch
git log --author="Nama"
```

Bab 4 — Branching & Merging

4.1 Branch Commands

```
git branch                  # lihat branch lokal
git branch -a               # lokal + remote
git switch -c nama-branch   # buat + pindah (cara modern)
git checkout nama-branch    # pindah ke branch ada
git branch -d nama-branch   # hapus (setelah merge)
```

4.2 Strategi Branching Tim

Branch	Tujuan	Aturan
main	Kode produksi stabil	Protected — tidak boleh push langsung
dev	Integrasi semua fitur	Selalu dalam kondisi berjalan
feature/xxx	Satu fitur/tugas	Buat dari dev, merge kembali ke dev
fix/xxx	Perbaikan bug	Buat dari dev atau main

4.4 Resolusi Conflict

Conflict terjadi ketika dua branch mengubah baris yang sama. Git tidak bisa memutuskan versi mana yang benar — harus diselesaikan manual.

```
<<<<<<< HEAD (branch aktif)
print("versi Anda")
=======
print("versi branch lain")
>>>>>>> feature/nama-fitur

# Solusi: edit file, hapus marker, pilih/gabungkan kode
# Lalu: git add nama-file && git commit
```

> **Perhatian:** Jangan commit file yang masih mengandung conflict marker <<<<<<. Ini akan merusak kode.

Kapan Conflict Terjadi?

Conflict tidak hanya muncul saat merge. Ada empat situasi yang bisa memicu conflict:

Perintah	Kapan Conflict Bisa Muncul
git merge	Dua branch mengubah baris yang sama (paling umum)
git pull	Teman mengubah file yang sama saat kamu sedang kerja lokal
git rebase	Conflict per-commit saat replay di atas branch target
git stash pop	Stash dan working dir mengubah bagian yang sama

Langkah-Langkah Resolve Conflict

Ikuti langkah-langkah berikut secara berurutan untuk menyelesaikan conflict dengan aman:

1	**Identifikasi file yang conflict** — jalankan git status. File conflict ditandai both modified
2	**Buka file di editor** — cari marker <<<<<<<. Edit isi file: pilih versi HEAD, versi branch lain, atau gabungkan keduanya. Hapus semua marker.
3	**Stage file yang sudah di-resolve** git add nama-file.py
4	**Selesaikan merge dengan commit** git commit (Git sudah menyiapkan pesan merge otomatis)

Resolve via VS Code (Direkomendasikan)

VS Code menampilkan UI khusus untuk conflict sehingga tidak perlu edit marker secara manual. Saat conflict terdeteksi, VS Code akan menampilkan empat tombol di atas setiap blok conflict:

Pilihan VS Code	Arti
Accept Current Change	Pakai versi kamu (HEAD) — buang versi branch lain
Accept Incoming Change	Pakai versi branch yang di-merge — buang versi kamu
Accept Both Changes	Gabungkan keduanya (kamu masih perlu edit manual hasilnya)
Compare Changes	Lihat perbedaan side-by-side sebelum memilih

Batalkan Merge yang Berantakan

Jika conflict terlalu kompleks dan kamu ingin kembali ke kondisi sebelum merge, gunakan perintah berikut:

```
# Batalkan merge yang sedang berjalan (kembali ke kondisi sebelum merge)
git merge --abort

# Batalkan rebase yang sedang berjalan
git rebase --abort
```

Tips Mencegah Conflict

Conflict lebih baik dicegah daripada diselesaikan. Terapkan kebiasaan berikut bersama tim:

#	Kebiasaan	Alasannya
1	Selalu git pull sebelum mulai kerja	Mulai dari kode yang paling update
2	Commit kecil dan sering	Konflik lebih kecil, lebih mudah diselesaikan
3	Satu orang, satu file atau modul	Hindari dua orang edit file yang sama bersamaan
4	Komunikasikan dengan tim file yang sedang dikerjakan	Cegah overlap pekerjaan sejak awal

Bab 5 — Remote Repository

5.2 Push & Pull

```
git push -u origin nama-branch    # push + set upstream
git push                          # push setelah upstream di-set
git pull                          # fetch + merge dari remote
git pull --rebase origin dev      # pull dengan rebase
```

5.3 Fetch vs Pull

	git fetch	git pull
Yang dilakukan	Download ke tracking branches saja	fetch + merge otomatis ke branch aktif
Ubah working dir?	Tidak	Ya (jika ada perubahan)
Kapan digunakan	Ingin lihat perubahan dulu	Langsung update branch lokal

Bab 6 — Kolaborasi Tim

6.1 Workflow Tim

Situasi	Tindakan
Mulai kerja	git pull origin dev — selalu update sebelum mulai
Mulai fitur baru	git switch -c feature/nama-fitur
Selama pengerjaan	Commit kecil dan sering. Satu commit = satu perubahan logis
Siap untuk review	git push → buat Pull Request ke dev
Setelah PR di-merge	git checkout dev && git pull
Hapus branch lama	git branch -d feature/nama-fitur

6.2 .gitignore

```
# Python & Data Science
__pycache__/  *.pyc  .env  venv/  .venv/
.ipynb_checkpoints
data/raw/  *.csv  *.xlsx
models/*.pkl  models/*.joblib
.DS_Store  Thumbs.db  .vscode/  .idea/
```

6.3 Git Stash

```
git stash                    # simpan perubahan sementara
git stash save "WIP: fitur login" # dengan label
git stash list               # lihat daftar stash
git stash pop                # kembalikan stash terakhir
```

Bab 7 — Undo & Recovery

```
# Batalkan perubahan sebelum commit
git restore nama-file.py        # undo edit di working dir
git restore --staged nama-file   # unstage file

# Batalkan commit yang sudah ada (AMAN — buat commit baru)
git revert HEAD

# Reset (BERBAHAYA jika sudah push)
git reset --soft HEAD~1         # batalkan commit, simpan di staging
git reset --hard HEAD~1         # batalkan commit + hapus perubahan
```

7.4 Panduan Kapan Menggunakan Apa

Situasi	Perintah	Aman?
Batal edit sebelum git add	git restore nama-file	✅ Ya
Unstage file	git restore --staged nama-file	✅ Ya
Perbaiki pesan commit terakhir	git commit --amend	✅ Jika belum push
Batalkan commit yang sudah push	git revert	✅ Selalu aman
Hapus commit terakhir, simpan perubahan	git reset --soft HEAD~1	⚠ Jika belum push
Hapus commit + semua perubahan	git reset --hard HEAD~1	❌ Berbahaya

Perhatian: Jangan gunakan git reset --hard pada commit yang sudah di-push ke shared branch.

Bab 8 — Good Practice: Gitmoji & Conventional Commits

Commit message yang baik adalah investasi jangka panjang. Conventional Commits dan Gitmoji adalah standar yang saling melengkapi untuk riwayat yang informatif dan konsisten.

8.1 Conventional Commits

```
# Format
<type>(<scope>): <description>

# Contoh
feat(auth): add login with Google OAuth
fix(api): handle null response from endpoint
docs(readme): update installation instructions
refactor(model): simplify preprocessing pipeline
```

Type	Kapan Digunakan
feat	Fitur baru
fix	Perbaikan bug
docs	Update dokumentasi
style	Formatting tanpa ubah logika
refactor	Restrukturisasi tanpa tambah fitur
test	Tambah atau perbaiki test
chore	Update tools, dependensi, konfigurasi
perf	Perbaikan performa
revert	Membatalkan commit sebelumnya

8.2 Gitmoji

```
# Format gabungan Gitmoji + Conventional Commits
:emoji: type(scope): deskripsi singkat

:sparkles: feat(dashboard): add real-time prediction chart
:bug: fix(model): correct feature scaling order
:memo: docs(report): add EDA summary week 4
:rocket: deploy(docker): update Dockerfile to python 3.11
```

Gitmoji	Code	Type	Penggunaan
✨	:sparkles:	feat	Fitur baru
🐛	:bug:	fix	Perbaikan bug
📝	:memo:	docs	Update dokumentasi / report
♻️	:recycle:	refactor	Refactor kode

Gitmoji	Code	Type	Penggunaan
🚀	:rocket:	deploy	Dockerfile, CI/CD, deployment
📊	:bar_chart:	data	Update dataset / EDA notebook
🤖	:robot:	model	Update atau tambah model ML
✅	:white_check_mark:	test	Tambah atau perbaiki test
🔧	:wrench:	chore	Konfigurasi, tools
🔥	:fire:	chore	Hapus kode atau file tidak perlu

8.3 Good vs Bad Commit Message

❌ Buruk	✅ Baik
fix stuff	:bug: fix(api): resolve null pointer in user endpoint
update	:memo: docs(readme): add Docker setup instructions
changes	:sparkles: feat(auth): implement JWT refresh token
final	:rocket: deploy(docker): add multi-stage build

8.4 Contoh Workflow Lengkap

```
git checkout dev && git pull origin dev
git switch -c feature/eda-titanic
# ... kerjakan tugas ...
git add notebooks/01-eda-titanic.ipynb
git commit -m ":bar_chart: data(eda): add EDA notebook Titanic"
git add reports/eda-titanic-report.md
git commit -m ":memo: docs(report): add EDA summary Titanic"
git push -u origin feature/eda-titanic
# Buka GitHub → Compare & pull request → merge ke dev
```

Bab 9 — Referensi Perintah Git

Setup & Init

Perintah	Deskripsi
git init	Inisialisasi repo baru
git clone <url>	Clone dari remote
git config --global user.name "..."	Set nama pengguna
git config --list	Lihat konfigurasi

Stage & Commit

Perintah	Deskripsi
git status	Lihat status file
git add <file>	Stage file
git add .	Stage semua
git commit -m "msg"	Commit dengan pesan
git commit --amend	Ubah commit terakhir

Branch & Merge

Perintah	Deskripsi
git branch	Lihat branch lokal
git branch -a	Lihat semua branch
git switch -c <nama>	Buat + pindah branch
git merge <branch>	Merge ke branch aktif
git branch -d <nama>	Hapus branch

Remote

Perintah	Deskripsi
git remote add origin <url>	Tambah remote
git push -u origin <branch>	Push pertama kali
git push	Push setelah upstream di-set
git pull	Fetch + merge dari remote
git fetch	Download tanpa merge

Undo

Perintah	Deskripsi
git restore <file>	Batalkan edit di working dir
git restore --staged <file>	Unstage
git revert <hash>	Buat commit baru yang membalik (aman)
git reset --soft HEAD~1	Batalkan commit, simpan di staging
git reset --hard HEAD~1	Batalkan commit + hapus perubahan

Bab 10 — Bibliografi

- Chacon, S. & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book
- Git Reference Manual. https://git-scm.com/docs
- GitHub Docs. https://docs.github.com
- Conventional Commits v1.0.0. https://www.conventionalcommits.org
- Gitmoji Guide. https://gitmoji.dev
- Learn Git Branching (Interaktif). https://learngitbranching.js.org
- Oh Shit, Git!? https://ohshitgit.com